

Flutter 开发学习资料完整版

目录

- 第一部分：环境搭建
 - 第二部分：Dart 语言基础
 - 第三部分：Flutter 核心概念
 - 第四部分：常用 Widget 详解
 - 第五部分：布局系统
 - 第六部分：状态管理
 - 第七部分：路由导航
 - 第八部分：网络请求与数据处理
 - 第九部分：本地存储
 - 第十部分：实战技巧与最佳实践
 - 附录：本项目代码结构解读
-

第一部分：环境搭建

1.1 安装 Flutter SDK

Linux 安装步骤

1. 下载 Flutter SDK：

```
cd ~/development
git clone https://github.com/flutter/flutter.git -b stable
```

2. 添加到 PATH：

```
export PATH="$PATH:$HOME/development/flutter/bin"
```

将上面这行添加到 `~/.bashrc` 或 `~/.zshrc` 中。

3. 运行 Flutter Doctor 检查环境：

```
flutter doctor
```

4. 根据提示安装缺少的依赖（如 Android SDK、Chrome 等）。

推荐开发工具

工具	说明	推荐度
VS Code + Flutter 插件	轻量级，启动快，插件丰富	★★★★★
Android Studio	功能全面，适合 Android 开发者	★★★★★

IntelliJ IDEA

企业级 IDE，大型项目适用



提示：VS Code 中安装 Flutter 和 Dart 插件后，可以获得代码补全、Widget Inspector、热重载等功能。

1.2 创建第一个 Flutter 项目

```
flutter create my_first_app
cd my_first_app
flutter run
```

这会生成一个标准的 Flutter 项目结构：

```
my_first_app/
├── lib/
│   └── main.dart          # 应用入口
├── android/              # Android 平台代码
├── ios/                  # iOS 平台代码
├── web/                  # web 平台代码
├── test/                 # 测试代码
├── pubspec.yaml          # 依赖配置文件
└── README.md
```

第二部分：Dart 语言基础

2.1 变量与数据类型

变量声明

```
// 使用 var 自动推断类型
var name = '张三';           // 推断为 String
var age = 25;                 // 推断为 int

// 显式声明类型
String greeting = '你好';
int count = 10;
double price = 9.99;
bool isActive = true;

// final - 运行时常量，只能赋值一次
final DateTime now = DateTime.now();

// const - 编译时常量
const double pi = 3.14159;

// 动态类型
dynamic anything = '可以是任何类型';
anything = 42; // 合法
```

注意： `final` 和 `const` 的区别：`const` 是编译时就必须确定的常量，`final` 是运行时确定且只能赋值一次。在 Flutter 中，经常使用 `const` 来优化 Widget 性能。

集合类型

```
// List (列表)
var fruits = ['苹果', '香蕉', '橙子'];
fruits.add('葡萄');
print(fruits.length); // 4
print(fruits[0]);      // '苹果'

// Map (键值对)
var user = {
  'name': '张三',
  'age': 25,
  'email': 'zhangsan@example.com'
};
print(user['name']);    // '张三'

// Set (集合, 元素唯一)
var numbers = {1, 2, 3, 3}; // 实际只有 {1, 2, 3}

// 泛型
List names = ['张三', '李四'];
Map scores = {'数学': 95, '英语': 88};
```

2.2 函数

```
// 基本函数
int add(int a, int b) {
  return a + b;
}

// 箭头函数 (单表达式)
int multiply(int a, int b) => a * b;
```

```
// 可选参数
void greet(String name, {String? greeting, int? age}) {
  print('${greeting ?? '你好'}, $name! 年龄: ${age ?? '未知'}');
}
greet('张三'); // 你好, 张三! 年龄: 未知
greet('张三', greeting: '早上好', age: 25); // 早上好, 张三! 年龄: 25

// 命名参数用 {} 包裹, 可以用 required 标记必填
void createUser({required String name, required int age}) {
  print('创建用户: $name, $age岁');
}

// 默认参数
void showMessage(String msg, {String level = 'info'}) {
  print('[$level] $msg');
}

// 函数作为参数 (高阶函数)
List filter(List list, bool Function(int) test) {
  return list.where(test).toList();
}
var even = filter([1, 2, 3, 4, 5], (n) => n % 2 == 0); // [2, 4]
```

2.3 类与对象

```
// 基本类
class Person {
  String name;
  int age;

  // 构造函数
  Person(this.name, this.age);
}
```

```
// 命名构造函数
Person.unknown() : name = '未知', age = 0;

// 方法
void introduce() {
  print('我是$name, 今年$age岁');
}

// Getter
bool get isAdult => age >= 18;

// 重写 toString
@override
String toString() => 'Person($name, $age)';
}

// 使用
var person = Person('张三', 25);
person.introduce(); // 我是张三, 今年25岁
print(person.isAdult); // true

// 继承
class Student extends Person {
  String school;
  Student(String name, int age, this.school) : super(name, age);

  @override
  void introduce() {
    super.introduce();
    print('就读于$school');
  }
}

// 抽象类
abstract class Animal {
```

```
void speak(); // 抽象方法，没有实现
}

class Dog extends Animal {
  @override
  void speak() => print('汪汪! ');
}

// Mixin (混入) - 实现多继承效果
mixin Flyable {
  void fly() => print('飞行中...');
}

mixin Swimmable {
  void swim() => print('游泳中...');
}

class Duck extends Animal with Flyable, Swimmable {
  @override
  void speak() => print('嘎嘎! ');
}

// Duck 可以 fly()、swim()、speak()
```

2.4 异步编程 (async/await)

```
// Future - 代表一个异步操作的结果
Future fetchUserData() async {
  // 模拟网络请求
  await Future.delayed(Duration(seconds: 2));
  return '用户数据';
}

// 使用 async/await
```



```
void loadData() async {
  print('开始加载...');
  String data = await fetchUserData(); // 等待异步结果
  print('数据: $data');
  print('加载完成');
}

// Future.then 链式调用
fetchUserData()
  .then((data) => print('数据: $data'))
  .catchError((error) => print('错误: $error'));

// Stream - 代表一系列异步事件
Stream countStream() async* {
  for (int i = 1; i <= 5; i++) {
    await Future.delayed(Duration(seconds: 1));
    yield i; // 产出一个值
  }
}

// 监听 Stream
countStream().listen((count) {
  print('计数: $count');
});

// try-catch 处理异步错误
Future safeLoadData() async {
  try {
    var data = await fetchUserData();
    print(data);
  } catch (e) {
    print('加载失败: $e');
  } finally {
    print('操作结束');
  }
}
```

```
}  
}
```

核心概念：Flutter 中大量使用 `Future` 和 `async/await` 处理异步操作，如网络请求、数据库读写、文件操作等。掌握异步编程是 Flutter 开发的基础。

2.5 扩展方法与泛型

```
// 扩展方法 - 给现有类添加新方法  
extension StringExtension on String {  
    String capitalize() {  
        return '${this[0].toUpperCase()}${substring(1)}';  
    }  
}  
print('hello'.capitalize()); // 'Hello'  
  
// 泛型  
class ApiResponse {  
    final int code;  
    final T? data;  
    final String? message;  
  
    ApiResponse({required this.code, this.data, this.message});  
  
    bool get isSuccess => code == 200;  
}  
  
var response = ApiResponse(code: 200, data: user);  
var errorResponse = ApiResponse(code: 500, message: '服务器错误');
```

第三部分：Flutter 核心概念

3.1 一切皆 Widget

Flutter 中**一切都是 Widget**。Widget 是 Flutter UI 的基本构建块，类似于乐高积木。每个 Widget 都描述了它在给定配置和状态下应该如何显示。

```
// 一个最简单的 Flutter 应用
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: '我的应用',
      home: Scaffold(
        appBar: AppBar(title: Text('首页')),
        body: Center(
          child: Text('你好, Flutter! '),
        ),
      ),
    );
  }
}
```

Widget 的两种类型

类型	特点	示例
<code>StatelessWidget</code>	无状态，创建后不会改变	Text、Icon、IconButton
<code>StatefulWidget</code>	有状态，可以在运行时改变	Checkbox、TextField、Slider

```
// StatelessWidget - 无状态组件
class Greeting extends StatelessWidget {
  final String name;

  const Greeting({Key? key, required this.name}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Text('你好, $name!');
  }
}

// StatefulWidget - 有状态组件
class Counter extends StatefulWidget {
  @override
  _CounterState createState() => _CounterState();
}

class _CounterState extends State {
  int _count = 0;

  void _increment() {
    setState(() { // 调用 setState 触发 UI 刷新
      _count++;
    });
  }

  @override
```

```
Widget build(BuildContext context) {  
  return Column(  
    children: [  
      Text('点击次数: $_count'),  
      ElevatedButton(  
        onPressed: _increment,  
        child: Text('加 1'),  
      ),  
    ],  
  );  
}
```

重要： `setState()` 会通知 Flutter 框架当前 State 已改变，需要重新调用 `build()` 方法更新 UI。不要在 `build()` 中调用 `setState()`，否则会死循环。

3.2 Widget 生命周期

`StatefulWidget` 的生命周期方法调用顺序：

1. `createState()` - 创建 State 对象
2. `initState()` - 初始化状态（只调用一次）
3. `didChangeDependencies()` - 依赖变化时调用
4. `build()` - 构建 UI（可能多次调用）
5. `didUpdateWidget()` - Widget 配置变化时调用
6. `setState()` - 触发重建
7. `deactivate()` - 从树中移除时调用
8. `dispose()` - 销毁时调用（释放资源）

```
class MyWidget extends StatefulWidget {  
  @override  
  _MyWidgetState createState() => _MyWidgetState();  
}  
  
class _MyWidgetState extends State {  
  late TextEditingController _controller;  
  
  @override  
  void initState() {  
    super.initState();  
    _controller = TextEditingController();  
    print('initState 被调用');  
  }  
  
  @override  
  void didChangeDependencies() {  
    super.didChangeDependencies();  
    print('didChangeDependencies 被调用');  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return TextField(controller: _controller);  
  }  
  
  @override  
  void dispose() {  
    _controller.dispose(); // 必须释放资源!  
    super.dispose();  
  }  
}
```

3.3 Key 的作用

Key 用于帮助 Flutter 识别 Widget，在列表中尤为重要：

```
// GlobalKey - 基于值的 Key
ListView(
  children: items.map((item) =>
    ListTile(
      key: GlobalKey(item.id), // 帮助 Flutter 识别列表项
      title: Text(item.name),
    )
  ).toList(),
)

// Key 的类型：
// - GlobalKey：基于值
// - UniqueKey：每次都是唯一的
// - ObjectKey：基于对象引用
```

第四部分：常用 Widget 详解

4.1 基础 Widget

Text - 文本

```
// 基本文本
Text('你好世界')

// 带样式的文本
Text(
  '这是一段很长的文本，会根据屏幕宽度自动换行显示。',
  style: TextStyle(
    fontSize: 18,
    fontWeight: FontWeight.bold,
    color: Colors.blue,
    letterSpacing: 1.5,
    height: 1.5, // 行高
  ),
  textAlign: TextAlign.center,
  maxLines: 3, // 最多显示 3 行
  overflow: TextOverflow.ellipsis, // 超出部分显示 ...
)

// 富文本（一段文字中混合不同样式）
RichText(
  text: TextSpan(
    text: '你好 ',
    style: TextStyle(color: Colors.black, fontSize: 16),
    children: [
      TextSpan(
```



```

        text: 'Flutter',
        style: TextStyle(color: Colors.blue, fontWeight: FontWeight.bold)
    ),
    TextSpan(text: ' 世界! '),
],
),
)

```

Container - 容器

```

// Container 是最常用的装饰容器
Container(
  width: 200,
  height: 100,
  padding: EdgeInsets.all(16),          // 内边距
  margin: EdgeInsets.symmetric(vertical: 10), // 外边距
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12), // 圆角
    boxShadow: [                          // 阴影
      BoxShadow(
        color: Colors.black12,
        blurRadius: 10,
        offset: Offset(0, 4),
      ),
    ],
    border: Border.all(color: Colors.grey.shade300), // 边框
    gradient: LinearGradient(              // 渐变
      colors: [Colors.blue, Colors.purple],
    ),
  ),
  child: Center(child: Text('卡片内容')),
)

```

Image - 图片

```
// 从网络加载图片
Image.network(
  'https://example.com/photo.jpg',
  width: 300,
  height: 200,
  fit: BoxFit.cover,      // 填充方式
  loadingBuilder: (context, child, loadingProgress) {
    if (loadingProgress == null) return child;
    return Center(child: CircularProgressIndicator());
  },
  errorBuilder: (context, error, stackTrace) {
    return Icon(Icons.error);
  },
)
```

```
// 从本地资源加载图片
Image.asset('assets/images/logo.png')
```

```
// BoxFit 常用值:
// BoxFit.cover    - 裁剪填满 (最常用)
// BoxFit.contain - 完整显示, 可能有留白
// BoxFit.fill     - 拉伸填满, 可能变形
// BoxFit.fitWidth - 宽度填满
// BoxFit.fitHeight - 高度填满
```

4.2 按钮 Widget

```
// ElevatedButton - 凸起按钮 (最常用)
ElevatedButton(
  onPressed: () => print('点击了'),
  child: Text('确定'),
```

```
style: ElevatedButton.styleFrom(
  primary: Colors.blue,
  onPressed: Colors.white,
  padding: EdgeInsets.symmetric(horizontal: 24, vertical: 12),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(8),
  ),
),
)

// TextButton - 文字按钮
TextButton(
  onPressed: () {},
  child: Text('取消'),
)

// OutlinedButton - 轮廓按钮
OutlinedButton(
  onPressed: () {},
  child: Text('查看详情'),
)

// IconButton - 图标按钮
IconButton(
  icon: Icon(Icons.favorite, color: Colors.red),
  onPressed: () {},
)

// FloatingActionButton - 浮动按钮
FloatingActionButton(
  onPressed: () {},
  child: Icon(Icons.add),
)

// 带图标的按钮
```

```
ElevatedButton.icon(  
  onPressed: () {},  
  icon: Icon(Icons.save),  
  label: Text('保存'),  
)
```

4.3 输入 Widget

```
// TextField - 文本输入框  
TextField(  
  controller: _textController,  
  decoration: InputDecoration(  
    labelText: '用户名',  
    hintText: '请输入用户名',  
    prefixIcon: Icon(Icons.person),  
    suffixIcon: IconButton(  
      icon: Icon(Icons.clear),  
      onPressed: () => _textController.clear(),  
    ),  
    border: OutlineInputBorder(  
      borderRadius: BorderRadius.circular(8),  
    ),  
    filled: true,  
    fillColor: Colors.grey.shade100,  
  ),  
  onChanged: (value) => print('输入: $value'),  
  onSubmitted: (value) => print('提交: $value'),  
)  
  
// Checkbox - 复选框  
Checkbox(  
  value: _isChecked,  
  onChanged: (bool? value) {
```

```
        setState(() {
          _isChecked = value ?? false;
        });
      },
    )

// Switch - 开关
Switch(
  value: _isDarkMode,
  onChanged: (bool value) {
    setState(() {
      _isDarkMode = value;
    });
  },
)

// DropdownButton - 下拉选择
DropdownButton(
  value: _selectedValue,
  items: ['选项1', '选项2', '选项3']
    .map((e) => DropdownMenuItem(value: e, child: Text(e)))
    .toList(),
  onChanged: (String? value) {
    setState(() {
      _selectedValue = value;
    });
  },
)
```

4.4 对话框

```
// 显示简单对话框
showDialog(
```

```
context: context,
builder: (context) => AlertDialog(
  title: Text('确认'),
  content: Text('确定要删除吗? '),
  actions: [
    TextButton(
      onPressed: () => Navigator.pop(context),
      child: Text('取消'),
    ),
    TextButton(
      onPressed: () {
        // 执行删除操作
        Navigator.pop(context);
      },
      child: Text('确定'),
    ),
  ],
),
);
```

// 底部弹出菜单

```
showModalBottomSheet(
  context: context,
  builder: (context) => Container(
    padding: EdgeInsets.all(16),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        ListTile(
          leading: Icon(Icons.camera),
          title: Text('拍照'),
          onTap: () {},
        ),
        ListTile(
          leading: Icon(Icons.photo),
```

```
        title: Text('从相册选择'),
        onTap: () {},
      ),
    ],
  ),
),
);

// Snackbar - 底部提示
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text('操作成功'),
    action: SnackBarAction(label: '撤销', onPressed: () {}),
    duration: Duration(seconds: 2),
  ),
);
```

4.5 列表与网格

```
// ListView - 垂直列表
ListView(
  children: [
    ListTile(
      leading: Icon(Icons.person),
      title: Text('张三'),
      subtitle: Text('在线'),
      trailing: Icon(Icons.chevron_right),
      onTap: () {},
    ),
    ListTile(
      leading: Icon(Icons.person),
      title: Text('李四'),
      subtitle: Text('离线'),
```

```
    ),  
  ],  
)  
  
// ListView.builder - 动态列表 (适合长列表, 会懒加载)  
ListView.builder(  
  itemCount: items.length,  
  itemBuilder: (context, index) {  
    return ListTile(  
      title: Text(items[index].name),  
    );  
  },  
)  
  
// GridView - 网格  
GridView.builder(  
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(  
    crossAxisCount: 2,           // 每行 2 个  
    crossAxisSpacing: 10,  
    mainAxisSpacing: 10,  
    childAspectRatio: 0.75,     // 宽高比  
  ),  
  itemCount: photos.length,  
  itemBuilder: (context, index) {  
    return Card(  
      child: Image.network(photos[index].url, fit: BoxFit.cover),  
    );  
  },  
)  
  
// 带分隔线的列表  
ListView.separated(  
  itemCount: items.length,  
  separatorBuilder: (context, index) => Divider(),  
  itemBuilder: (context, index) {
```



```
        return ListTile(title: Text(items[index]));  
      },  
    )  
  )  
}
```

第五部分：布局系统

5.1 线性布局

Row - 水平排列

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceEvenly, // 主轴对齐  
  crossAxisAlignment: CrossAxisAlignment.center,    // 交叉轴对齐  
  children: [  
    Icon(Icons.star, color: Colors.red),  
    Text('评分'),  
    Text('4.5'),  
  ],  
)
```

Column - 垂直排列

```
Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  crossAxisAlignment: CrossAxisAlignment.start,  
  children: [  
    Text('标题', style: TextStyle(fontSize: 24)),  
    SizedBox(height: 8), // 间距  
    Text('副标题'),  
    SizedBox(height: 16),  
    ElevatedButton(onPressed: () {}, child: Text('按钮')),  
  ],  
)
```

主轴对齐方式 (MainAxisAlignment)

值	效果
start	靠左/顶部对齐
end	靠右/底部对齐
center	居中
spaceBetween	两端对齐，中间等距
spaceAround	每个元素两侧等距
spaceEvenly	所有间距完全相等

5.2 层叠布局

```
// Stack - 层叠布局
Stack(
  children: [
    // 底层
    Image.network('https://example.com/photo.jpg', fit: BoxFit.cover),
    // 半透明遮罩
    Container(color: Colors.black38),
    // 上层文字
    Positioned(
      bottom: 16,
      left: 16,
      child: Text(
        '标题',
        style: TextStyle(color: Colors.white, fontSize: 24),
      ),
    ),
  ],
)
```

```
    ),  
    // 右上角徽章  
    Positioned(  
      top: 8,  
      right: 8,  
      child: Container(  
        padding: EdgeInsets.symmetric(horizontal: 8, vertical: 4),  
        decoration: BoxDecoration(  
          color: Colors.red,  
          borderRadius: BorderRadius.circular(12),  
        ),  
        child: Text('NEW', style: TextStyle(color: Colors.white, fontSize:  
      ),  
    ),  
  ],  
)
```

5.3 弹性布局

```
// Expanded - 占据剩余空间  
Row(  
  children: [  
    Icon(Icons.menu),  
    Expanded( // 占据剩余所有空间  
      child: TextField(  
        decoration: InputDecoration(  
          hintText: '搜索...',  
          border: InputBorder.none,  
        ),  
      ),  
    ),  
    Icon(Icons.search),  
  ],  
)
```

```
    ],  
  )  
  
  // Flexible - 按比例分配  
  Row(  
    children: [  
      Flexible(  
        flex: 2,  
        child: Container(color: Colors.red, height: 50),  
      ),  
      Flexible(  
        flex: 1,  
        child: Container(color: Colors.blue, height: 50),  
      ),  
      Flexible(  
        flex: 1,  
        child: Container(color: Colors.green, height: 50),  
      ),  
    ],  
  )  
  // 红色占 2/4, 蓝色占 1/4, 绿色占 1/4
```

5.4 Padding 与 SizedBox

```
  // Padding - 添加内边距  
  Padding(  
    padding: EdgeInsets.all(16),           // 四边 16  
    child: Text('四周都有内边距'),  
  )  
  
  Padding(  
    padding: EdgeInsets.symmetric(horizontal: 20, vertical: 10),  
    child: Text('水平 20, 垂直 10'),  
  )
```

```
)

Padding(
  padding: EdgeInsets.only(left: 16, top: 8),
  child: Text('只有左边和上边有内边距'),
)

// SizedBox - 固定尺寸的盒子 / 间距
SizedBox(width: 16),           // 水平间距 16
SizedBox(height: 20),          // 垂直间距 20
SizedBox(
  width: 100,
  height: 100,
  child: CircularProgressIndicator(), // 固定 100x100 的区域
)

// Wrap - 自动换行的布局
Wrap(
  spacing: 8,
  runSpacing: 8,
  children: [
    Chip(label: Text('Flutter')),
    Chip(label: Text('Dart')),
    Chip(label: Text('Widget')),
    Chip(label: Text('布局')),
    Chip(label: Text('状态管理')),
  ],
)
```

第六部分：状态管理

6.1 setState - 最基础的方式

```
class CounterPage extends StatefulWidget {  
  @override  
  _CounterPageState createState() => _CounterPageState();  
}  
  
class _CounterPageState extends State {  
  int _count = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Center(  
        child: Text('$count', style: TextStyle(fontSize: 48)),  
      ),  
      floatingActionButton: FloatingActionButton(  
        onPressed: () => setState(() => _count++),  
        child: Icon(Icons.add),  
      ),  
    );  
  }  
}
```

适用场景：简单页面内的局部状态管理。当页面复杂或状态需要跨页面共享时，应考虑更高级的方案。

6.2 Provider

```
// 1. 定义数据模型
class CounterModel extends ChangeNotifier {
  int _count = 0;
  int get count => _count;

  void increment() {
    _count++;
    notifyListeners(); // 通知监听者刷新
  }
}

// 2. 在顶层提供数据
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterModel(),
      child: MyApp(),
    ),
  );
}

// 3. 在子 Widget 中使用
class CounterPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // 读取数据
    final counter = context.watch();

    return Scaffold(
      body: Center(
        child: Text('${counter.count}', style: TextStyle(fontSize: 48)),
      ),
    );
  }
}
```



```
floatingActionButton: FloatingActionButton(  
  onPressed: () => context.read().increment(),  
  child: Icon(Icons.add),  
),  
);  
}  
}
```

// 只在部分 UI 刷新 (优化性能)

```
class CountDisplay extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    // select 只监听 count 属性变化  
    final count = context.select(  
      (model) => model.count,  
    );  
    return Text('$count');  
  }  
}
```

6.3 BLoC (Business Logic Component)

// 本项目使用的状态管理方案

// 1. 定义 Event

```
abstract class CounterEvent {}  
class IncrementEvent extends CounterEvent {}  
class DecrementEvent extends CounterEvent {}
```

// 2. 定义 State

```
class CounterState {  
  final int count;  
  CounterState({this.count = 0});  
}
```

```
}

// 3. 定义 BLoC
class CounterBloc extends Bloc {
  CounterBloc() : super(CounterState()) {
    on((event, emit) {
      emit(CounterState(count: state.count + 1));
    });
    on((event, emit) {
      emit(CounterState(count: state.count - 1));
    });
  }
}

// 4. 在 Widget 中使用
BlocProvider(
  create: (_) => CounterBloc(),
  child: BlocBuilder(
    builder: (context, state) {
      return Text('${state.count}');
    },
  ),
)

// 发送事件
context.read.add(IncrementEvent());
```

BLoC vs Provider: BLoC 适合复杂业务逻辑，事件驱动，便于测试；Provider 适合简单状态共享，使用更简洁。本项目采用 BLoC 模式。

6.4 状态管理方案对比

方案	复杂度	适用场景	学习成本
setState	低	单个 Widget 的局部状态	★
InheritedWidget	中	框架级数据共享	★★
Provider	低	中小型应用的全局状态	★★
BLoC	中	大型应用的业务逻辑	★★★
Riverpod	中	Provider 的改进版	★★
GetX	低	快速开发，全功能框架	★★

第七部分：路由导航

7.1 基本导航

```
// 跳转到新页面
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => DetailPage()),
);

// 返回上一页
Navigator.pop(context);

// 替换当前页面（不能返回）
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => HomePage()),
);

// 跳转并传递数据
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => DetailPage(itemId: 42),
  ),
);

// 返回并传递数据
Navigator.pop(context, '选择的结果');
```

7.2 命名路由

```
// 1. 注册路由
MaterialApp(
  routes: {
    '/': (context) => HomePage(),
    '/detail': (context) => DetailPage(),
    '/settings': (context) => SettingsPage(),
  },
);

// 2. 使用命名路由跳转
Navigator.pushNamed(context, '/detail');

// 3. 传递参数
Navigator.pushNamed(
  context,
  '/detail',
  arguments: {'id': 42, 'title': '详情'},
);

// 4. 接收参数
class DetailPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final args = ModalRoute.of(context)!.settings.arguments as Map;
    return Text('ID: ${args['id']}');
  }
}
```

7.3 GoRouter (推荐)

```
// 本项目使用的路由方案
final router = GoRouter(
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => HomePage(),
    ),
    GoRoute(
      path: '/detail/:id',
      builder: (context, state) {
        final id = state.params['id']!;
        return DetailPage(id: id);
      },
    ),
    ShellRoute(
      builder: (context, state, child) {
        return MainLayout(child: child); // 带底部导航栏
      },
      routes: [
        GoRoute(path: '/home', builder: (_, __) => HomePage()),
        GoRoute(path: '/media', builder: (_, __) => MediaPage()),
        GoRoute(path: '/settings', builder: (_, __) => SettingsPage()),
      ],
    ),
  ],
);

// 使用
context.go('/detail/42');           // 跳转
context.push('/detail/42');         // 压入导航栈
context.pop();                      // 返回
```

第八部分：网络请求与数据处理

8.1 HTTP 请求

```
// 使用 http 包（需要添加依赖：http）
import 'package:http/http.dart' as http;
import 'dart:convert';

// GET 请求
Future<List> fetchUsers() async {
  final response = await http.get(
    Uri.parse('https://api.example.com/users'),
    headers: {'Authorization': 'Bearer $token'},
  );

  if (response.statusCode == 200) {
    List data = json.decode(response.body);
    return data.map((json) => User.fromJson(json)).toList();
  } else {
    throw Exception('加载失败');
  }
}

// POST 请求
Future createUser(User user) async {
  final response = await http.post(
    Uri.parse('https://api.example.com/users'),
    headers: {'Content-Type': 'application/json'},
    body: json.encode(user.toJson()),
  );
}
```

```
if (response.statusCode == 201) {  
    return User.fromJson(json.decode(response.body));  
} else {  
    throw Exception('创建失败');  
}  
}
```

8.2 JSON 序列化

```
// 手动序列化  
class User {  
    final int id;  
    final String name;  
    final String email;  
  
    User({required this.id, required this.name, required this.email});  
  
    // 从 JSON 创建对象  
    factory User.fromJson(Map json) {  
        return User(  
            id: json['id'],  
            name: json['name'],  
            email: json['email'],  
        );  
    }  
  
    // 转换为 JSON  
    Map toJson() {  
        return {  
            'id': id,  
            'name': name,  
            'email': email,  
        };  
    }  
}
```



```
}  
}  
  
// 使用 json_serializable 自动生成 (推荐)  
// 需要添加依赖: json_annotation, json_serializable, build_runner  
import 'package:json_annotation/json_annotation.dart';  
part 'user.g.dart';  
  
@JsonSerializable()  
class User {  
  final int id;  
  final String name;  
  final String email;  
  
  User({required this.id, required this.name, required this.email});  
  
  factory User.fromJson(Map json) => _$UserFromJson(json);  
  Map toJson() => _$UserToJson(this);  
}  
  
// 运行代码生成: dart run build_runner build
```

第九部分：本地存储

9.1 SharedPreferences

```
// 存储简单的键值对数据
import 'package:shared_preferences/shared_preferences.dart';

// 写入
Future saveSettings() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('username', '张三');
  await prefs.setInt('theme_mode', 1);
  await prefs.setBool('is_dark_mode', true);
}

// 读取
Future loadSettings() async {
  final prefs = await SharedPreferences.getInstance();
  String? username = prefs.getString('username');
  int themeMode = prefs.getInt('theme_mode') ?? 0;
  bool isDarkMode = prefs.getBool('is_dark_mode') ?? false;
}
```

9.2 SQLite 数据库

```
// 使用 sqflite 包
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
```

```
class DatabaseHelper {
  static Database? _database;

  Future get database async {
    _database ??= await _initDatabase();
    return _database!;
  }

  Future _initDatabase() async {
    String path = join(await getDatabasesPath(), 'my_app.db');
    return openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            content TEXT,
            created_at TEXT DEFAULT CURRENT_TIMESTAMP
          )
        ''');
      },
    );
  }

  // CRUD 操作

  Future insertNote(Map note) async {
    final db = await database;
    return await db.insert('notes', note);
  }

  Future>> getNotes() async {
    final db = await database;
    return await db.query('notes', orderBy: 'created_at DESC');
```

```
}

Future updateNote(int id, Map note) async {
  final db = await database;
  return await db.update('notes', note, where: 'id = ?', whereArgs: [id]
}

Future deleteNote(int id) async {
  final db = await database;
  return await db.delete('notes', where: 'id = ?', whereArgs: [id]);
}
}
```

第十部分：实战技巧与最佳实践

10.1 响应式设计

```
// 获取屏幕尺寸
MediaQuery.of(context).size.width
MediaQuery.of(context).size.height

// 根据屏幕宽度选择布局
Widget build(BuildContext context) {
  final width = MediaQuery.of(context).size.width;

  if (width > 1200) {
    return DesktopLayout(); // 桌面端
  } else if (width > 600) {
    return TabletLayout(); // 平板
  } else {
    return MobileLayout(); // 手机
  }
}

// 使用 LayoutBuilder
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth > 600) {
      return Row(children: [Sidebar(), Content()]);
    } else {
      return Column(children: [Content()]);
    }
  }
)
```

```
},  
)
```

10.2 性能优化

```
// 1. 使用 const 构造函数  
// 好的做法  
const Text('不会改变的文字')  
const EdgeInsets.all(16)  
const Icon(Icons.star)  
  
// 2. 避免在 build 中创建对象  
// 不好  
Widget build(BuildContext context) {  
  final style = TextStyle(fontSize: 16); // 每次 build 都创建  
  return Text('Hello', style: style);  
}  
  
// 好 (使用 const 或提为静态变量)  
static const _style = TextStyle(fontSize: 16);  
  
// 3. 使用 ListView.builder 而不是 ListView  
// 不好 (一次性创建所有子 widget)  
ListView(children: items.map((e) => ItemWidget(e)).toList())  
  
// 好 (懒加载, 只创建可见的子 widget)  
ListView.builder(  
  itemCount: items.length,  
  itemBuilder: (context, index) => ItemWidget(items[index]),  
)  
  
// 4. 拆分 widget 减少重建范围  
// 不好 - 整个页面重建
```

```
class MyPage extends StatefulWidget {  
    ...  
}  
  
// 好 - 只重建需要变化的部分  
class MyPage extends StatelessWidget {  
    @override  
    Widget build(BuildContext context) {  
        return Column(  
            children: [  
                const Header(),           // 不会重建  
                const SizedBox(height: 20), // 不会重建  
                CounterSection(),          // 只有这部分重建  
            ],  
        );  
    }  
}
```

10.3 调试技巧

```
// 1. print 调试  
print('变量值: $variable');  
  
// 2. debugPrint (限制输出频率, 避免被截断)  
debugPrint('大量输出内容...');  
  
// 3. Flutter Inspector  
// VS Code 中按 Ctrl+Shift+P 搜索 "Flutter Inspector"  
// 可以可视化查看 widget 树  
  
// 4. Widget Inspector 标记  
Widget build(BuildContext context) {  
    return Container(  

```

```
// debugPaintSizeEnabled = true 可显示布局边界
child: Text('Hello'),
);
}

// 5. 断点调试
// 在 VS Code 中点击行号左侧设置断点
// 按 F5 启动调试模式

// 6. Flutter DevTools
// 运行 flutter run 后在浏览器中打开 DevTools
// 可以查看 widget 树、性能、网络、日志等
```

10.4 常用命令

命令	说明
<code>flutter create myapp</code>	创建新项目
<code>flutter run</code>	运行应用（热重载按 r，热重启按 R）
<code>flutter build apk</code>	构建 Android APK
<code>flutter build ios</code>	构建 iOS 应用
<code>flutter build linux</code>	构建 Linux 桌面应用
<code>flutter build web</code>	构建 Web 应用
<code>flutter pub get</code>	安装依赖
<code>flutter pub add 包名</code>	添加依赖

<code>flutter analyze</code>	代码分析
<code>flutter test</code>	运行测试
<code>flutter clean</code>	清理构建缓存
<code>flutter doctor</code>	检查环境配置

附录：本项目代码结构解读

A.1 项目目录结构

```
AdvanceMediaKB-FR/
├─ lib/                                # Dart 源代码
│   ├─ main.dart                       # 应用入口
│   ├─ bridge/native/                 # FFI 桥接层（连接 Rust/C++ 后端）
│   │   ├─ api/                       # API 接口定义
│   │   │   ├─ album.dart             # 相册相关接口
│   │   │   ├─ media.dart             # 媒体文件接口
│   │   │   ├─ tag.dart               # 标签接口
│   │   │   ├─ note.dart              # 笔记接口
│   │   │   └─ search.dart            # 搜索接口
│   │   └─ settings.dart              # 设置接口
│   │   └─ models.dart                # 数据模型
│   │   └─ native_library.dart         # 原生库加载
│   └─ core/                           # 核心模块
│       ├─ design_system/              # 设计系统
│       │   ├─ app_theme.dart          # 主题配置
│       │   └─ components.dart         # 通用组件
│       └─ i18n/                        # 国际化
│           └─ app_localizations.dart  # 中文文案
│       └─ navigation/                  # 路由配置
│           └─ app_router.dart          # GoRouter 路由定义
│       └─ permissions/                 # 权限管理
├─ functionality/                       # 业务逻辑层 (BLoC)
│   ├─ album/                           # 相册 BLoC
│   ├─ media/                           # 媒体 BLoC
│   ├─ note/                             # 笔记 BLoC
│   └─ tag/                             # 标签 BLoC
```

```
| | └─ home/ # 应用全局 BLoC
| └─ ui/ # UI 界面层
|   └─ home/home_screen.dart # 首页
|   └─ media/media_screen.dart # 媒体页
|   └─ album/album_screen.dart # 相册页
|   └─ tag/tag_screen.dart # 标签页
|   └─ search/search_screen.dart # 搜索页
|   └─ settings/settings_screen.dart # 设置页
|   └─ note/ # 笔记页
|   └─ viewer/ # 查看器
└─ native/ # C++ 后端代码
  └─ src/db/ # SQLite 数据库
  └─ src/api/ # 后端 API 实现
└─ rust/ # Rust 代码（可选后端）
└─ docs/ # 文档
└─ pubspec.yaml # 依赖配置
└─ .gitignore # Git 忽略规则
```

A.2 架构模式

本项目采用 **BLoC 模式**，将代码分为三层：

层级	职责	对应目录
UI 层	负责显示界面，接收用户输入	lib/ui/
业务逻辑层	处理业务逻辑，管理状态	lib/functionality/
数据层	与后端通信，数据持久化	lib/bridge/

A.3 数据流向

用户操作 → UI 发送 Event → BLoC 处理 Event

↓

BLoC 调用 API → 后端返回数据

↓

BLoC 更新 State → UI 响应 State 变化重建

A.4 官方资源汇总

- Flutter 官方文档（中文）：<https://docs.flutter.cn/>
- Flutter Cookbook：<https://docs.flutter.cn/cookbook>
- Dart 官方文档：<https://dart.cn/>
- Dart 语言之旅：<https://dart.cn/guides/language/language-tour>
- Flutter Widget 目录：<https://docs.flutter.cn/ui/widgets>
- Flutter BLoC 库文档：<https://bloclibrary.dev/>
- Flutter 中文社区：<https://flutter-io.cn/>

文档生成时间：2026 年 7 月 | AdvanceMediaKB 项目